Course Project 1: AI-Powered Image Editing Platform

Module 5 Assignment: Editing + Versioning (Week 2 Submission) | **Student:** Muskan Dawar

🔗 **GitHub Repository:**
https://github.com/muskan-1301/ai-image-editor

1. Executive Summary & Architecture Overview

This project delivers a portfolio-ready MVP for an AI-first image editing platform. Rather than requiring complex manual tools, users interact with their photos using natural language instructions (e.g. "*add a calm blue ocean with gentle waves in the background*", "*change background to sunset*", "*oil painting style*"). The platform integrates multimodal Vision LLMs with a generative synthesis pipeline and an SQLite-backed versioning system.

2. Deliverables Checklist & Verification

Course Deliverable	Implementation Component	Status
Functional Edit Interface with LLM + Vision	Streamlit UI (`app/main.py`) using Gemini Vision (`gemini-2.5-flash`) to analyze images and execute edits.	Complete
Quick-Action Preset Buttons	8 one-click presets (Ocean, Sunset, Palm trees, Oil painting, Winter, Night lights, B&W, Studio lighting) auto-filling prompt box.	Complete
Version History Logic	SQLite relational schema (`images` and `versions`) tracking every edit (`v001`, `v002`...) with timestamps and prompts.	Complete
Thumbnails & Multi-Version Slider	Automatic 256x256 thumbnail generation and an interactive slider to compare and download past versions.	Complete
Prompt Handling Scripts	Modular pipeline in `app/ai_clients.py` for image captioning, multimodal planning, and semantic vector embeddings.	Complete
Natural Language Semantic Search	Embedding search using `gemini-embedding-001` and cosine similarity ranking.	Complete

3. API Process & Selection Rationale

A. Multimodal Vision Pipeline

Traditional diffusion models struggle when asked to edit existing personal photos because they lack context of the original scene. To resolve this, our application uses a **two-stage multimodal pipeline**:

1. **Stage 1: Vision Grounding (LLM Inspection):** When an image is uploaded or edited, its raw bytes are processed via `google.genai` and sent to **Gemini 2.5 Flash**. The LLM identifies the primary subjects, lighting, clothing, and composition.

2. **Stage 2: Contextual Edit Formulation:** Gemini takes the user's conversational prompt and generates a specialized scene synthesis prompt that preserves the subject while inserting the new elements.
3. **Stage 3: Generative Synthesis & Storage:** The synthesized image is saved as a discrete PNG version in `data/versions/`, indexed in SQLite, and rendered with an auto-generated thumbnail in the history timeline.

B. Semantic Vector Embeddings for Natural Language Search

Search queries are encoded into 3072-dimensional vector vectors via **`gemini-embedding-001`**. The cosine similarity is calculated against all stored captions in real time:

```
Cosine Similarity = (A · B) / (||A|| * ||B||)
```

4. Prompt Engineering Templates

Template 1: Vision Auto-Captioning

```
"Describe this image in 2–3 sentences. Be specific about colours, objects, setting, and mood. Output plain text only."
```

Purpose: Produces objective, descriptive metadata optimal for semantic vector search without conversational noise.

Template 2: Multimodal Edit Planning

```
"The user wants to edit this picture with the instruction: '{prompt}'.
Describe the final edited scene in 1 concise, vivid sentence for an AI image generator.
Keep the core subject/style from the original image and incorporate the edit.
Return only the scene description."
```

Purpose: Prevents visual hallucination by forcing the LLM to anchor the edit to the existing subject matter.

5. Database Schema (SQLite: `data/image_editor.db`)

```
CREATE TABLE images (
    id TEXT PRIMARY KEY, filename TEXT, filepath TEXT,
    thumbnail TEXT, caption TEXT, embedding TEXT, uploaded_at TEXT, provider TEXT
);

CREATE TABLE versions (
    id TEXT PRIMARY KEY, image_id TEXT REFERENCES images(id),
    version_num INTEGER, filepath TEXT, thumbnail TEXT,
    prompt TEXT, provider TEXT, model TEXT, created_at TEXT, notes TEXT
);
```

6. Instructions to Run the Application

```
1. Clone repository:
   git clone https://github.com/muskan-1301/ai-image-editor.git
```

```
   cd ai-image-editor

2. Install dependencies:
   pip install -r requirements.txt

3. Set up environment:
   cp .env.example .env
   # Add your GEMINI_API_KEY in .env

4. Run the Streamlit application:
   streamlit run app/main.py
   # Application opens at http://localhost:8501
```